Tyk

Tyk Onboarding

Sr. Customer Solutions Architect

Agend

a

Maintenance

01 **Working with data**
Understanding Tyk objects
Managing databases

02 **Migration between environments**
Running A Dashboard Dev to Prod

03 **Upgrading Tyk**
Understanding upgrade strategies
Steps to deploy and configure Tyk

04 **Performance Tuning**
Performance Tune Tyk

05 **Monitoring**
Using OTel for monitoring

Tyk

Module 6

Working with Data

Understanding data and Tyk objects
Database housekeeping

Tyk

Tyk Database Overview

Tyk uses two different databases - Redis and MongoDB:

- Redis stores data necessary for the Gateway
- MongoDB/PostgreSQL stores data necessary for the Dashboard

Deployments without a Dashboard do not require MongoDB/Postgres

Databases deployed based on vendor best-practices
Compatible SaaS services can be used

Redis

In-memory data store and caching system, often used as a fast and scalable key-value database

Tyk

Redis - data storage

Used by the Gateway for storage of:

- API Keys
- Cached responses
- Temporary analytics
- Emergency configuration (for worker Gateways in MDCB deployments)
- Uptime test allocation
- Liveness probe data

Redis - data storage

Used by the Dashboard for storage of:

- Dashboard user sessions
- Dashboard API keys
- SSO nonces
- Certificates
- Gateway registrations

Redis - data storage

Tyk uses prefixes and namespaces in Redis key names:

- Standard API key: `apikey-<KEY_HASH>`
- OAuth API key: `oauth-data.<API_ID>.oauth-clientid.<OAUTH_CLIENT_ID>`
- Dashboard API key: `tyk-admin-api-<DASHBOARD_API_KEY>`
- Dashboard user session: `tyk-admin-api-<SESSION_ID>`
- Certificate: `cert-raw-<CERTIFICATE_ID>`

Redis - Messaging

Publisher/subscriber messaging for the Gateway and Dashboard:

- Zero configuration
- Hot reloads
- Distributed rate limiter

Messages can be signed for authenticity using a digital signature:

- Gateway configuration: set `allow_insecure_configs` to `false` and `public_key_path` to path of public key
- Dashboard configuration: set `private_key_path` to path of private key

Redis - Security

From Redis 6, TLS is an optional built-in feature:

- Must be compiled from source to enable TLS
- Use 3rd party applications, such as Spiped
- SaaS offerings provide secure endpoints
- Set `redis.use_ssl` to `true` in Gateway configuration
- Set `redis_use_ssl` to `true` in Dashboard configuration

Tyk

Redis - Persistence

Redis has two approaches for persistence:

- Redis Database Backup (RDB), is a point-in-time snapshots of the database
- Append Only File (AOF), is a log of all write operations

Both approaches can be used simultaneously

RDB is the traditional database backup snapshot:

- Use the `SAVE` or `BGSAVE` commands
- Creates file called `dump.rdb`
- Can be scheduled

Redis - Persistence

AOF allows for instances to automatically re-populate data after restarting:

- Set appendonly to true in the Redis configuration file
- Operations which modify data are written to AOF every second
- AOF is rewritten to optimise the file and reduce size

Redis - High Availability

Redis has two approaches for high availability - Sentinel and Cluster:

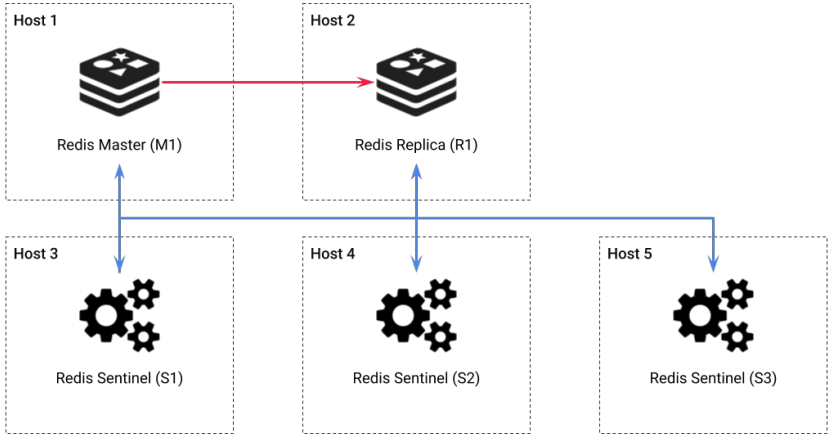
- Requires a minimum of three separate hosts for high availability
- All Redis instances must be able to communicate with each other
- Data is replicated from a Master instance to one or more Replica instances
- Asynchronous replication with an eventually consistent system
- A majority of Sentinels/Masters is required for failover to occur
- A Replica is promoted to Master during failover
- IP and port remapping can cause issues

Redis - Sentinel

Sentinel instances are responsible for discovery, monitoring and failover
Minimum of three Sentinel instances are required:

- Majority required for failover

Sentinels can share same host as data bearing instances



Redis - Cluster

Redis Cluster uses a sharded data set:

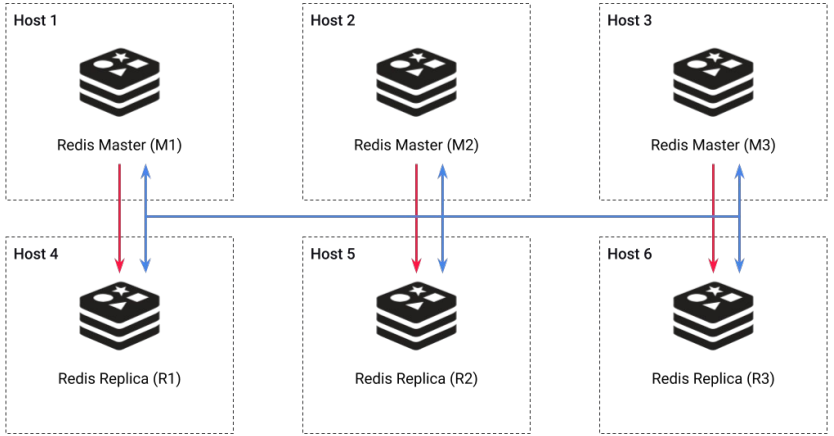
- Horizontal scaling option

Minimum of 3 Master instances required:

- Master majority required for failover

Cluster message bus used for intra-instance communication

Replica can share hosts with Master that does not cover same shard



Tyk

Redis - Error Handling

In the event of Redis failure, the Gateway will temporarily disable Redis-based functionality:

- Key lookups - APIs which use authentication based on Redis key lookups will return errors
- Rate limits and Quotas
- Redis Pub/Sub messaging
- Recording analytics data

Once Redis availability is restored, Gateway functionality will resume

Gateway will report Redis status as `fail` via health check endpoint

Redis - Sizing disk/RAM

API Keys:

- Depends on number of Keys stored
- Typical API Key is approximately 1KB
- Example: 1,000 API Keys @ 1KB = 1MB

Cached responses:

- Depends on number of responses cached per second, average response size and cache lifetime
- Example: 1,000 responses cached per second @ 20KB with 60 seconds lifetime = 1.2GB

Redis - Sizing disk/RAM

Temporary analytics:

- Depends on number of requests per second, pump purge delay and average combined request and response size (if recording detailed analytics)
- Recording detailed analytics can significantly impact storage requirement
- Base analytics record is approximately 1KB
- Example: 1,000 requests per second @ 50KB average combined size plus 1KB base record data, with a pump purge delay of 10 seconds = 510MB

Redis - Sizing disk/RAM

Emergency configuration:

- Depends on number and size of API Definitions and Policies
- API Definition ranges from 15KB to more than 100KB
- Policy ranges from 1KB to more than 5KB
- Example: 100 API Definitions @ 50KB and 500 Policies @ 2KB = 6MB

Dashboard sessions:

- Depends on the number of concurrent Dashboard sessions
- Typical Dashboard user session object is 1KB
- Example: 10 concurrent Dashboard sessions @ 1KB = 10KB

Tyk

Redis - Sizing disk/RAM

Configure the operating system to allow memory to be overcommitted:

- Forked process needs to duplicate data
- Additional RAM required for `BGSAVE` backup process
- Set `vm.overcommit_memory` to `1` in `/etc/sysctl.conf` to allow Redis process to be forked without requiring double the memory

Redis - Sizing CPU

Redis executes commands in a single-threaded manner:

- Multi-core CPUs do not offer much advantage
- Higher API throughput requires higher frequency processor
- Use cluster approach to increase performance

Redis bottleneck is not normally CPU, usually either memory or network

MongoDB

NoSQL database that uses a document-oriented model, storing data in flexible, JSON-like BSON format, and is designed for scalability and high performance.

Tyk

MongoDB - Data Storage

Used by the Dashboard for storage of all Dashboard-managed data, with exception of API Keys and Certificates

Used by the Pump for storage of analytics data

MongoDB - Data Storage

Data stored in different collections within the database:

- API Definitions: `tyk_apis`
- Policies: `tyk_policies`
- Organisations: `tyk_organisations`
- Portal pages: `portal_pages`
- Organisation-specific analytics: `z_tyk_analyticz_<ORGANISATION_ID>`

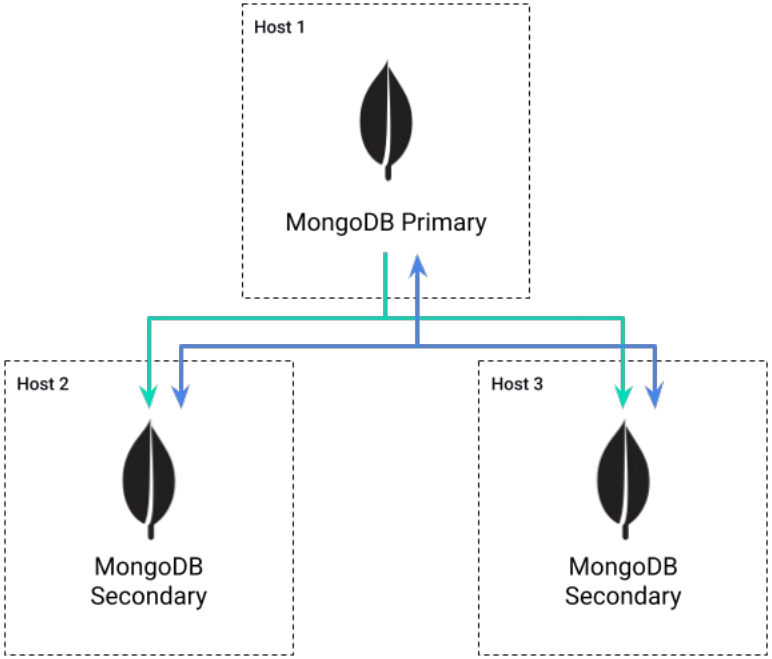
MongoDB - High Availability

MongoDB uses Replica Sets for high availability:

- Requires a minimum of three separate hosts for high availability
- All members of Replica Set must be able to communicate with each other
- Election occurs if Primary becomes inaccessible
- Members can have different priorities, to determine the new Primary
- A majority of Replica Set members are required for an election
- Non-voting members: cannot vote, but do hold data
- Arbiters: can vote, but do not hold data

MongoDB - Replica Set

Data replicated from Primary to one or more Secondary instances
Heartbeat maintained between all instances



Tyk

MongoDB - Sizing Disk

System configuration:

- Depends on the number of API Definitions, Policies etc
- Not likely to be of significant size

Non-aggregated analytics

- Depends on the number of requests and payload size for detailed recording
- Basic analytics record is less than 2KB
- Detailed recording adds size of request and response payload
- Example: 1m requests @ 2KB without detailed recording = 2GB
- Example: 1m requests @ 2KB with detailed recording @ 50KB = 52GB

MongoDB - Sizing Disk

Aggregated analytics:

- Depends on the number of APIs, API versions, Tags, Keys, OAuth Clients, Endpoints, Geolocations and API Error Codes
- Only active data is recorded
- Does not vary based on API throughput
- Aggregated into hourly chunks
- 1KB per item stored in the aggregated record
- Example: 1000 active API keys + 100 active APIs + 100 versions = 1.2MB per hourly record (29MB per day, or 2.6GB for 90 days)

MongoDB - Managing Size

Utilise storage strategies to manage database size:

- Capped collection: limits the size of analytics collections
- Time-to-live index: limits the ages of analytics records
- MongoDB CLI: MongoDB CLI query to remove analytics records

MongoDB - Sizing RAM

MongoDB RAM is used for storing indexes and commonly accessed data - known as the *working set*

The working set typically includes:

- API Definitions
- Policies
- Aggregated report data
- Unlikely to require more than 100-200MB

MongoDB - Sizing RAM

Non-aggregated analytics data collections can require large indexes:

- Approximately 160MB per 1 million analytics documents

Aggregated indexes are usually small as there are far fewer documents

MongoDB - Sizing CPU

MongoDB is capable of utilising multiple CPU cores:

- Storage engine benefits from multiple threads
- Recommended for MongoDB to have access to at least two single-core CPUs, or one multi-core CPU

Ideally one thread available for every concurrent operation:

- More Dashboards, Pumps and MDCBs requires more available threads

PostgreSQL

Open-source relational database management system (RDBMS) known for its extensibility, compliance with SQL standards, and support for complex queries and transactions.

Tyk

Postgres - Data Storage

Used by the Dashboard for storage of all Dashboard-managed data, with exception of API Keys and Certificates

Used by the Pump for storage of analytics data

Postgres - Data Storage

Data stored in different tables within the database:

- API Definitions: `tyk_apis`
- Policies: `tyk_policies`
- Organisations: `tyk_organisations`
- Portal pages: `portal_pages`
- Analytics: `tyk_analytics`

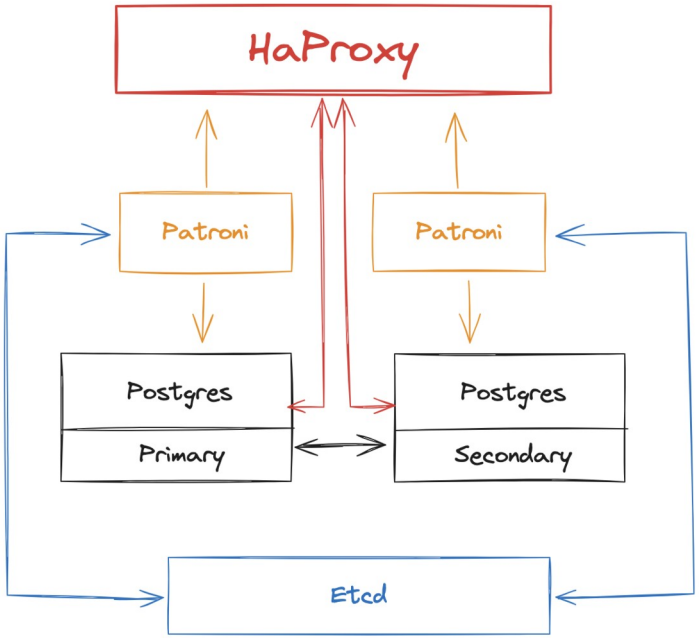
Postgres - High Availability

- Postgres has 2 main ways of replication
  - Physical replication
  - Logical replication
- Physical replication
  - Maintains a full copy of the entire data of a cluster
  - Entire set of data on the primary server is copied to the replica which acts as a standby node
  - Good for disaster recovery
- Logical replication
  - Replicates data objects and their changes based on a unique identifier
  - Copies database objects in a row-based model as opposed to physical replication
  - Good for clustering due to performance

PostgreSQL

Using a combination of streaming replication, Patroni & Etcd for automated failover, and HAProxy for load balancing thus achieving HA:

- Continuous Synchronization by streaming write-ahead logs (WAL) from primary to standby nodes
- HAProxy monitors health of servers (Primary & Standby)
- Distributes incoming database connections
- Etcd acts as a distributed key-value store for configuration management
- Patroni manages failover and PostgreSQL cluster management, monitors health of servers & checking status & connectivity
- Orchestrates standby promotion if there are issues detected on primary server



Postgres - Sizing Disk

System configuration:

- Depends on the number of API Definitions, Policies etc
- Not likely to be of significant size

Non-aggregated analytics

- Depends on the number of requests and payload size for detailed recording
- Basic analytics record is less than 2KB
- Detailed recording adds size of request and response payload
- Example: 1m requests @ 2KB without detailed recording = 2GB
- Example: 1m requests @ 2KB with detailed recording @ 50KB = 52GB

Postgres - Sizing Disk

Aggregated analytics:

- Depends on the number of APIs, API versions, Tags, Keys, OAuth Clients, Endpoints, Geolocations and API Error Codes
- Only active data is recorded
- Does not vary based on API throughput
- Aggregated into hourly chunks
- 1KB per item stored in the aggregated record
- Example: 1000 active API keys + 100 active APIs + 100 versions = 1.2MB per hourly record (29MB per day, or 2.6GB for 90 days)

Postgres - Managing Size

- Postgres does not have in-built functionality to cap collections
- Create a regular table and implementing custom logic to manage the size and rotation of the data
- `pgcron`
- Time based retention
  - Use a regular table and periodically delete older records based on a timestamp column
- Size based retention
  - Implement a custom logic to track the size of the table and delete older records when the table reaches a certain size
- Table Sharding
  - Tyk splits creates a new table on a per-day basis

Tyk

Postgres - Sizing RAM

MongoDB RAM is used for a number of processes in Postgres

- Shared Buffers
- Maintenance Work Memory
  - Sorting and Index Operations
- Connection-related Memory

Postgres - Sizing CPU

Postgres is capable of utilising multiple CPU cores:

- Storage engine benefits from multiple threads
- Recommended for Postgres to have access to at least two single-core CPUs, or one multi-core CPU

Ideally one thread available for every concurrent operation:

- More Dashboards, Pumps and MDCBs requires more available threads

Backing-up and Restoring

Explore methods of backing up and restoring Databases

Tyk

Backup & Restore - Redis

- Backup Process:
  - Use `SAVE` or `BGSAVE` commands for snapshot backups.
  - Syntax: `SAVE` (synchronous) or `BGSAVE` (asynchronous).
- Backup Options:
  - Snapshot Backup: Entire dataset at a point in time.
  - Append-Only File (AOF): Logs changes for point-in-time recovery.
- Scheduled Backups:
  - Schedule regular backups using cron jobs or third-party tools.
- Restoration Process:
  - Use `BGREWRITEAOF` for AOF file replay.
  - Restart Redis with the snapshot file for snapshot backups.
- Persistence Options:
  - Choose between RDB (Snapshot) and AOF (Append-Only File) persistence.
  - Configure persistence in the Redis configuration file.

Backup & Restore - MongoDB

- Backup Process:
  - Utilize `mongodump` command.
  - Syntax: `mongodump --host <hostname> --port <port> --out <backup_directory>`
- Backup Options:
  - Full Backup: Entire database.
  - Specific Database: `--db <database_name>`.
  - Collection: `--collection <collection_name>`.
- Automated Backups:
  - Set up periodic backups using tools like MongoDB Atlas or cron jobs.
- Restoration Process:
  - Use `mongorestore` command.
  - Syntax: `mongorestore --host <hostname> --port <port> --db <database_name> <backup_directory>`

Backup & Restore - Postgres

- Backup Process:
  - Use `pg_dump` for logical backups.
  - Syntax: `pg_dump -h <hostname> -p <port> -U <username> -d <database_name> -f <backup_file>`
- Backup Options:
  - Full Database: `--format=custom or --format=directory`.
  - Specific Table: `--table <table_name>`.
  - Compression: `-F c` for custom format with compression.
- Point-in-Time Recovery (PITR):
  - Archive WAL (Write-Ahead Logging) files for continuous backups.
  - Use `pg_basebackup` or tools like Barman for PITR.
- Automated Backups:
  - Implement periodic backups with tools like `pg_cron` or scheduling in cron jobs.
- Restoration Process:
  - Use `pg_restore` for logical restoration.
  - Syntax: `pg_restore -h <hostname> -p <port> -U <username> -d <database_name> <backup_file>`

Hands-On Workshop

Diving in databases

Backing up and restoring databases

Tyk

Module 7

Migration between Environments

Understanding data and Tyk objects
Database housekeeping

Tyk

Moving APIs - via Dashboard

Use the Export functionality of the Dashboard to download the API definition as JSON and import it into your new installation

- In `tyk_analytics.conf` change `enable_duplicate_slugs` to `true` to retain API ID

1. From the API Designer, select your API.
2. Click EXPORT
3. Save and rename the JSON file
4. In your new environment, click IMPORT API
5. Select the From Tyk Definition tab and paste the contents of the JSON file into the code editor and click GENERATE API

This will now import the API Definition into your new environment, if you have kept the API ID in the JSON document as is, the ID will remain the same

- Doing a manual migration using the Dashboard is useful for moving individual APIs
- Suitable for developer workflows without a VCS e.g GitHub

Moving Policies - via Dashboard

Not as easy as moving API definitions

Requires working with the Dashboard API to first retrieve the policies, and then modifying the document to reinsert them in your new environment

1. Preparation
    a. tyk.conf
        i. Edit `policies.allow_explicit_policy_id` to `true`
    b. tyk_analytics.conf
        i. Edit `allow_explicit_policy_id` to `true`
        ii. Set `enable_duplicate_slugs` to `true`
2. Get your Policy JSON file via Dashboard API
3. Edit the file and enter the value of `_id` to `id` and save
4. POST request to new environment using Dashboard API

Moving APIs - via Tyk Sync

Tyk Sync is a command-line tool and Go library designed for synchronizing API definitions and Security Policies from a Git repository or file system into Tyk.

- Versioning and Sync:
  - Enables versioning of Tyk configurations to Git or files.
  - Facilitates one-way synchronization from Git or files to Tyk.
- Workflow:
  - Developers configure and test APIs locally.
  - Utilizes commands like tyk-sync dump to convert APIs to transportable format.
  - Follows Git practices, creating Pull Requests for peer review.
  - Approved changes trigger deployment pipeline with commands like tyk-sync sync.
- Features:
  - Works with Tyk Gateway and Tyk Dashboard installations.
  - Supports dump, sync, update, and publish commands.
  - Specialized support for Git integration.

Does not work with Keys, specifically designed for APIs and Policies.

Moving APIs - via Tyk Sync

- Command Overview:
  - Dump: Extracts APIs and Policies into a directory for backup or transfer.
  - Publish: Publishes API definitions from Git to Tyk Gateway or Dashboard.
  - Sync: Synchronizes API Gateway with contents of a GitHub repository.
  - Update: Updates matching APIs or Policies in the target without creating new ones.
- Command Usage:
  - tyk-sync dump: Extracts and creates a spec file for syncing.
  - tyk-sync publish: Publishes API definitions from Git to Tyk.
  - tyk-sync sync: Synchronizes Tyk Gateway with a GitHub repository.
  - tyk-sync update: Updates existing APIs or Policies.
- Prerequisites:
  - Dashboard and Gateway configurations must meet specific criteria for ID matching.
- Installation:
  - Available via Docker and Packagecloud.
  - Docker commands provided for installation and usage.

Moving Policies - via Tyk Sync

- Example 1: Transfer from One Tyk Dashboard to Another:
  - tyk-sync dump: Extracts data from a Tyk Dashboard.
  - Git commands: Add, commit, and push changes.
  - tyk-sync sync: Restores data from GitHub to another Tyk Dashboard.

- Example 2: Dump a Specific API:
  - tyk-sync dump: Extracts a specific API from a Tyk Dashboard.

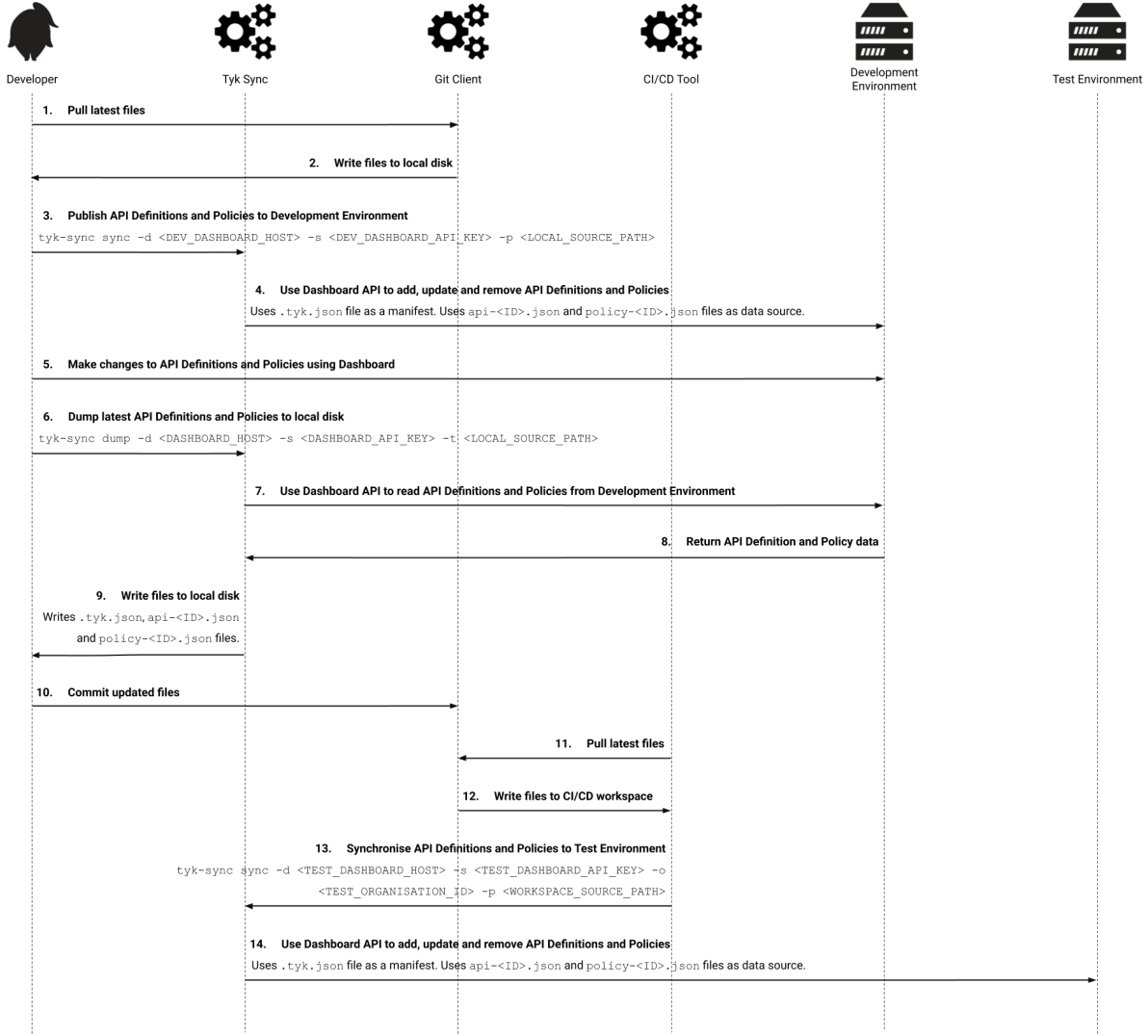
- Example 3: Check Tyk Sync Version:
  - tyk-sync version: Displays the current Tyk Sync version.

- Example 4: Import Tyk Example into Dashboard:
  - tyk-sync examples: Lists available examples.
  - tyk-sync examples show: Displays details of a specific example.
  - tyk-sync examples publish: Publishes an example into the Dashboard.

Tyk Sync streamlines API management, versioning, and synchronization workflows.

Tyk Sync

Example CI/CD workflow



Hands-On Workshop

Exploring Tyk-Sync

Tyk

Module 7

Upgrading Tyk

Understanding data and Tyk objects
Database housekeeping

Backing up Tyk

Essential to have a fresh backup, especially before changes or upgrades.

- Configuration Files Backup:
  - Regularly backup config files for all relevant components.
  - Use a version control system, such as Git.
- Config Files per Component:
  - Tyk Gateway: tyk.conf
  - Tyk Pump: pump.conf
  - Tyk Dashboard (Self-Managed): tyk_analytics.conf
  - MDCB: tyk_sink.conf
  - Hybrid Tyk Gateway: tyk.hybrid.conf
- Certificates Directory:
  - Backup certificates and keys defined in config files.
- Middleware Directory for Custom Plugins:
  - Regularly back up the middleware directory for custom plugins.
- Backup databases
  - Use best practice from official documentation

Upgrading Tyk

Regardless of your deployment choice (Linux, Docker, Kubernetes), we recommend the following upgrade process:

- Backup your Tyk deployment
- Get/update the latest binary (i.e. update the docker image name in the command, Kubernetes manifest or values.yaml of Helm chart or get the latest packages with `apt get`)
- Use individual deployment best practices for a rolling update (in local, non-shared, non-production environments simply restart the gateway)
- Check the log to see that the new version is used and that the gateway is up and running
- Check that the gateway is healthy using the open /hello endpoint of the Gateway API.

Upgrading Tyk - Linux

- Components installation order in a production environment
  - In a production environment, where we recommend installing the Dashboard, Gateway and Pump on separate machines, you should upgrade components in the following sequence:
    - Tyk Dashboard
    - Tyk Gateway
    - Tyk Pump

- Tyk is compatible with a blue-green or rolling update strategy.

- For a single-machine installation, follow the instructions below for your operating system.
- Ubuntu Upgrade
  - Use apt to update and upgrade as you would normally do with other apps.
- RHEL Upgrade
  - Example for release v5.0.0
    - sudo yum upgrade tyk-dashboard-5.0.0

Use the exact version to avoid upgrading other unrelated packages. You can find the package you want in the Packageclou

Hands-On Workshop

Upgrading Tyk Deployment

Tyk